Advanced Routing

Failover without Scripting

Introduction

Let us suppose that we have several WAN links, and we want to monitor, whether the Internet is accessible through each of them. The problem can be everywhere.

If your VPN cannot connect - then there's no problem, your default route with **gateway=that-vpn-connection** will be inactive.

If your ADSL modem is down - then **check-gateway=ping** is on stage, and no problem again.

But what if your modem is up, and telephone line is down? Or one of your ISP has a problem inside it, so traceroute shows only a few hops - and then stops...

Some people use NetWatch tool to monitor remote locations. Others use scripts to periodically ping remote hosts. And then disable routes or in some other way change the behaviour of routing.

But RouterOS facilities allow us to use only */ip routes* to do such checking - no scripting and netwatch at all!

Implementation

Basic Setup

Let's suppose that we have two uplinks: **GW1**, **GW2**. It can be addresses of ADSL modems (like *192.168.1.1* and *192.168.2.1*), or addresses of PPP interfaces (like *pppoe-out1* and *pptp-out1*). Then, we have some policy routing rules, so all outgoing traffic is marked with **ISP1** (which goes to **GW1**) and **ISP2** (which goes to **GW2**) marks. And we want to monitor **Host1** via **GW1**, and **Host2** via **GW2** - those may be some popular Internet websites, like Google, Yahoo, etc.

First, create routes to those hosts via corresponding gateways:

```
/ip route
add dst-address=Host1 gateway=GW1 scope=10
add dst-address=Host2 gateway=GW2 scope=10
```

Now we create rules for **ISP1** routing mark (one for main gateway, and another one for failover):

```
/ip route
add distance=1 gateway=Host1 routing-mark=ISP1 check-gateway=ping
add distance=2 gateway=Host2 routing-mark=ISP1 check-gateway=ping
```

Those routes will be resolved recursively (see Manual:IP/Route#Nexthop_lookup), and will be active only if **HostN** is pingable.

Then the same rules for **ISP2** mark:

```
/ip route
add distance=1 gateway=Host2 routing-mark=ISP2 check-gateway=ping
add distance=2 gateway=Host1 routing-mark=ISP2 check-gateway=ping
```

Multiple host checking per Uplink

If **Host1** or **Host2** in #Basic Setup fails, corresponding link is considered failed too. For redundancy, we may use several hosts per uplink: let's monitor **Host1A** and **Host1B** via **GW1**, and **Host2A** and **Host2B** via **GW2**. Also, we'll use double recursive lookup, so that there were fewer places where **HostN** is mentioned.

As earlier, first we need routes to our checking hosts:

```
/ip route
add dst-address=Host1A gateway=GW1 scope=10
add dst-address=Host1B gateway=GW1 scope=10
add dst-address=Host2A gateway=GW2 scope=10
add dst-address=Host2B gateway=GW2 scope=10
```

Then, let's create destinations to "virtual" hops to use in further routes. I'm using *10.1.1.1* and *10.2.2.2* as an example:

```
/ip route
add dst-address=10.1.1.1 gateway=Host1A scope=10 target-scope=10 check-
gateway=ping
add dst-address=10.1.1.1 gateway=Host1B scope=10 target-scope=10 check-
gateway=ping
add dst-address=10.2.2.2 gateway=Host2A scope=10 target-scope=10 check-
gateway=ping
add dst-address=10.2.2.2 gateway=Host2B scope=10 target-scope=10 check-
gateway=ping
```

And now we may add default routes for clients:

```
/ip route
add distance=1 gateway=10.1.1.1 routing-mark=ISP1
add distance=2 gateway=10.2.2.2 routing-mark=ISP1
add distance=1 gateway=10.2.2.2 routing-mark=ISP2
add distance=2 gateway=10.1.1.1 routing-mark=ISP2
```

Workaround 1

In ROS versions at least up to 4.10 there's a bug, and if your ethernet interface goes down (for example, your directly connected ADSL modem is powered off) and then brings up, recursive routes are not recalculated (or something) and all traffic still goes via another uplink. As a workaround, additional rules for each **HostN** may be used. When adding them, all is recalculated correctly:

```
/ip route
add dst-address=Host1 type=blackhole distance=20
add dst-address=Host2 type=blackhole distance=20
```